

Edge-Compute Pacing & Feedback Node

System & Firmware Design Document

Students: Sean Balbale, Zoe Davidow & Kayla Walsh
Program: B.S. ECE / B.S. CS, Trinity College (Class of 2027)
Faculty Advisor: Prof. Kenneth A. Kousen
Concept Summary: April 28, 2026
This Revision: September 2026 — adds hardware/ECE design detail
and revises the sensing model from power- to HR-based

1 Abstract

This project develops an embedded edge-compute node that gives endurance athletes — specifically rowers and cyclists — real-time fatigue feedback with no phone and no cloud in the loop. The device pairs with a wireless heart-rate chest strap (BLE, with an ANT+ fallback path if required), runs a physiologically motivated depletion/recovery model directly on a resource-constrained microcontroller, and expresses the result through an on-board LED array and buzzer. The contribution spans two disciplines: an **ECE** contribution in sensor interfacing, power budgeting, and concurrent real-time firmware on a strict memory and power envelope, and a **CS** contribution in the discretization and validation of a continuous-time physiological model running on fixed-point, resource-constrained hardware.

2 Problem Statement

Commercial cycling and rowing head units display raw telemetry (watts, cadence, heart rate, split) but perform no predictive computation. Athletes must self-interpret numbers while under physiological stress, a cognitively expensive task that degrades pacing decisions. Smartphone and cloud offload solve the compute problem but introduce connectivity dependency, latency, and battery vulnerability — unacceptable in competitive race settings.

Can a real-time fatigue model be implemented with sub-100 ms latency on a bare-metal or RTOS microcontroller, with sufficient numerical precision to provide actionable pacing feedback — entirely offline, and driven by sensor hardware that is realistically available to a rower or cyclist without a \$400+ power meter?

3 Sensing & Fatigue Model

3.1 From Power-Based to HR-Based Sensing

The original concept used the W' balance model, driven by an athlete's instantaneous power output. In practice, the vast majority of rowers and club/collegiate cyclists do not own a power meter (\$400–\$1500), while heart-rate chest straps are inexpensive (\$50–\$100) and already owned by most endurance athletes. To make the device usable by an actual team rather than only athletes with

power-meter-equipped bikes, the primary sensor input is changed to heart rate.

Decision: Primary fatigue input changes from power (W) to heart rate (bpm).

Rationale: Sensor availability and cost across the target user base (teammates, club athletes) rather than sensor precision. This is an access/equity decision, not a claim that HR is a better signal than power.

This has a real algorithmic consequence, documented for the advisor and for the written thesis: the Monod-Scherrer/Skiba W' model is defined and validated in terms of power against Critical Power. Heart rate is not power. It lags true effort by tens of seconds to minutes, does not register short anaerobic surges the way power does, and drifts with heat, hydration, and accumulated fatigue independent of instantaneous output. Sections 3.2 and 3.3 below keep the validated model as the literature reference case and present the HR-driven model as an explicitly *adapted, unvalidated* analog — which becomes a research question in its own right (Section 6.1) rather than a weakened version of the original claim.

3.2 Reference Case: The W' Balance Model

Two athlete-specific parameters govern the reference model:

- CP — Critical Power (W), the highest sustainable steady-state output.
- W' — the finite anaerobic energy reservoir (J) above CP .

$$W'_{\text{bal}}(t) = W' - \int_0^t [P(\tau) - CP]^+ d\tau \quad (1)$$

$$\frac{dW'_{\text{bal}}}{dt} = \begin{cases} -(P(t) - CP) & P(t) \geq CP \quad (\text{depletion}) \\ \frac{W' - W'_{\text{bal}}(t)}{\tau_{W'}} & P(t) < CP \quad (\text{recovery}) \end{cases} \quad (2)$$

$$\tau_{W'}(t) = 546 e^{-0.01 [CP - P(t)]} + 316 \quad [\text{s}] \quad (3)$$

This remains the literature-grounded case used to validate the discretization approach (Section 3.5) whenever a power meter is available for bench testing.

3.3 Adapted Case: HR-Analog Depletion Model

The primary, deployed model substitutes heart rate for power and lactate-threshold heart rate ($LTHR$) for Critical Power, keeping the same depletion/recovery structure:

$$H'_{\text{bal}}(t) = H' - \int_0^t [\text{HR}(\tau) - LTHR]^+ d\tau \quad (4)$$

$$\frac{dH'_{\text{bal}}}{dt} = \begin{cases} -(\text{HR}(t) - LTHR) & \text{HR}(t) \geq LTHR \\ \frac{H' - H'_{\text{bal}}(t)}{\tau_{H'}} & \text{HR}(t) < LTHR \end{cases} \quad (5)$$

where H' is a normalized reservoir size (not a physical energy quantity — there is no literature basis for an anaerobic joule-equivalent in HR terms) and $\tau_{H'}$ reuses the functional form of Eq. (3) with $LTHR$ in place of CP , pending empirical re-fitting in Phase 2. This substitution is stated plainly as a design assumption to be tested, not asserted as physiologically validated.

3.4 Threshold Calibration ($LTHR$, HR_{max} , HR_{rest})

Three athlete-specific constants are required and must be enterable in the field (Section 4.4):

- $LTHR$ — primary threshold for Eq. (5). Estimated via a guided on-device field test (last 20 minutes of a 30-minute time-trial effort, averaged), with manual entry as an override for athletes with a known lab-tested value.
- HR_{max} , HR_{rest} — used for Karvonen %HRR normalization in the calibration UI and for sanity-bounding sensor readings (reject implausible HR samples).

3.5 Discrete-Time Implementation

Both models share one parameterized Euler update, so the same firmware path handles the power case (bench validation) and the HR case (deployed):

$$X'_{\text{bal}}[n] = \begin{cases} X'_{\text{bal}}[n-1] - (S[n] - T) \Delta t & S[n] \geq T \\ X'_{\text{bal}}[n-1] + \frac{X' - X'_{\text{bal}}[n-1]}{\tau_{X'}[n]} \Delta t & S[n] < T \end{cases} \quad (6)$$

where $(S, T, X') = (P, CP, W')$ for the reference case or $(HR, LTHR, H')$ for the deployed case, $\Delta t = 1\text{ s}$ (matching the BLE HR notification interval), and $X'_{\text{bal}}[0] = X'$. Fixed-point arithmetic only; well within the Cortex-M4F budget at 64 MHz.

3.6 LED Feedback Mapping

$$k_{\text{lit}}[n] = \left\lfloor N \cdot \frac{X'_{\text{bal}}[n]}{X'} \right\rfloor, \quad \text{hue}[n] = \begin{cases} \text{green} & k_{\text{lit}}/N > 0.50 \\ \text{amber} & 0.20 < k_{\text{lit}}/N \leq 0.50 \\ \text{red} & k_{\text{lit}}/N \leq 0.20 \end{cases} \quad (7)$$

A buzzer interrupt fires when $X'_{\text{bal}}[n] < 0.10 \cdot X'$.

4 Hardware Architecture & Component Selection

4.1 Microcontroller

Decision: Nordic nRF52840 (Adafruit Feather nRF52840 Express for prototyping).

Rationale: Native dual-radio silicon (BLE 5 and ANT, on the same die) means the sensor-protocol decision below is a *firmware* choice, not a board respin. The Feather's built-in USB and LiPo charge circuitry also removes a discrete charger IC from the BOM. Cortex-M4F @ 64 MHz, 256 kB RAM, 1 MB flash — ample headroom for the Euler-update compute load, an I2C OLED driver, and FreeRTOS.

4.2 Wireless Sensor Protocol: BLE vs. ANT+

	BLE-only (Arduino/CircuitPython)	ANT+ (S340 SoftDevice)
Toolchain	Stays on Adafruit’s Arduino core, current firmware plan unchanged	Requires leaving Arduino for the nRF5 SDK / nRF Connect SDK; free ANT+ Adopter registration required to obtain the S340 hex
Strap compatibility	Covers any dual-mode strap (Polar H10, Garmin HRM-Dual/Pro, Wahoo TICKR — confirmed to broadcast BLE + ANT+ simultaneously)	Required only for ANT+-only hardware (rare among current chest straps)
Multi-athlete environments	Standard HR Service scan, connect to strap’s stored MAC address	15 configurable channels, better suited to crowded-field broadcast scenarios (e.g. a regatta with dozens of straps active)

Decision: Default to BLE-only; keep ANT+ as a documented fallback path, not a Phase 1 requirement.

Rationale: The straps realistically available to the team broadcast both protocols simultaneously, so BLE-only satisfies Phase 1 without the SDK migration cost. The ANT+ path (Section 4.2, right column) is retained as a designed-for option if a strap without BLE turns up, or if channel behavior in a crowded regatta start line makes ANT+ preferable — this is an open item, see Section 10.

4.3 Power System

Decision: 3.7 V LiPo pouch cell (JST-PH, 2000 mA h) instead of an 18650 cell with a separate BMS/charger module.

Rationale: The Feather’s onboard charge IC and battery-voltage monitoring already expect exactly this battery type and connector — adding a bare 18650 would mean sourcing and wiring a separate TP4056-class charger and physical cell holder for no functional benefit at this power budget.

Subsystem	Est. avg. current	Duty	Notes
nRF52840 core (sensor sampling + Euler compute)	5 mA	continuous	Cortex-M4F active during 1 Hz compute tick, sleeps between ticks
BLE connection (HR notifications @ 1 Hz)	1.5 mA	continuous	Connection interval tuned for the 1 s sensor cadence, not BLE’s fastest possible rate
NeoPixel strip (8 px, typical amber/green states)	15 mA	continuous	Worst case (8 px full white) is ~144 mA; normal operation stays well under this
OLED (calibration menu only)	15 mA	intermittent	Off during a session; averages under 1 mA over a full ride/row

Buzzer (threshold alerts)	30 mA	intermittent	Averages under 0.5 mA over a session
Estimated average draw	~ 23 mA		
Estimated runtime on	~ 85 h		Far exceeds the 6 h target
2000 mA h			

These are first-pass estimates from component datasheets, not bench measurements — Phase 1 hardware bring-up includes a runtime validation pass under real load (Section 8). Given the wide margin above the 6 h target, a smaller 500 mA h–1200 mA h cell is worth evaluating once real draw is measured, trading unused runtime for a smaller, lighter handlebar/hull-mount enclosure.

4.4 User Interface Hardware

- **Output:** WS2812B NeoPixel stick (8 px) for the primary X'_{bal} readout; piezo buzzer for the low-reserve alert.
- **Calibration input:** three 6 mm tactile buttons (Up / Down / Select) plus a 0.91-inch 128×32 I2C OLED (STEMMA QT), added specifically because entering a 3-digit $LTHR$ value via LED blink-count feedback is error-prone in the field. The OLED shares the I2C bus with no other peripheral, so it adds two GPIO pins total.
- **Config mode:** Select cycles $LTHR \rightarrow HR_{\text{max}} \rightarrow HR_{\text{rest}} \rightarrow \text{save/exit}$; Up/Down adjust the active field with accelerating auto-repeat on a held press. Buttons are inert during an active session to prevent accidental input mid-effort.

4.5 USB Connector

Decision: Add a USB Type-C breakout board (downstream configuration) wired to the Feather’s onboard Micro-USB connector, rather than using the stock Micro-USB port directly or an external Micro-B-to-C adapter cable.

Rationale: The Feather’s native USB already handles both charging and full data (serial console, DFU flashing) — the only gap is that its physical connector is Micro-USB. An adapter cable would close that gap with zero rework, but the breakout gives a cleaner result: it has the 5.1 k Ω CC pulldown resistors needed for reliable 5 V/1.5 A negotiation built in (avoiding the charger-compatibility question an unpowered adapter cable carries), and it panel-mounts flush into the 3D-printed enclosure wall instead of leaving an internal cable loop.

The nRF52840’s USB D+/D− lines are not broken out to any 0.1" header pin on the Feather — they run only to the pads of the onboard Micro-USB connector. Implementation is therefore a piggyback solder job, not a header-wire connection: four fine-gauge (30 AWG) wires are tacked directly onto the Micro-USB connector’s VBUS, D+, D−, and GND legs (the 5th, ID, pin is unused) and routed to the matching pads on the breakout board. Desoldering the Micro-USB connector outright is deliberately avoided, since lifting a pad on a fine-pitch SMT part is a real risk without hot-air rework equipment. Continuity should be bench-verified with a multimeter before final enclosure assembly, since the joints are not inspectable once sealed.

4.6 Mechanical / Form Factor

Handlebar mount (cycling) or hull-mount (rowing shell), 3D-printed in-house. Rowing introduces a splash/spray exposure the original cycling-only form factor did not need to fully account for; enclosure sealing around the button, OLED, and USB-C cutouts is called out as an open item (Section 10).

5 Firmware & Software Architecture

5.1 RTOS Task Structure

FreeRTOS with four tasks on defined priorities and circular-buffer IPC: **sensor** (BLE HR notification handling), **compute** (Eq. (6) Euler tick), **output** (NeoPixel/buzzer drive), and **UI** (button debounce and OLED refresh during calibration). The UI task is lowest priority and never blocks the sensor-to-LED path, preserving the 100 ms end-to-end latency target from the original problem statement.

5.2 Sensor Pairing

- **BLE:** scan for the standard Heart Rate Service (UUID 0x180D); connect to the first match; store the strap’s MAC address in flash. Subsequent boots connect directly to the stored address rather than re-scanning, falling back to a fresh scan only on a connection timeout. This avoids picking up a nearby teammate’s strap during team practice or a race start.
- **ANT+ (if implemented per Section 4.2):** wildcard search on HR profile (device type 120) once, then lock to the discovered device number for all subsequent searches.
- Both paths expose a manual re-pair trigger (button hold) for swapping straps.

5.3 Language & Validation

C/C++ against the nRF5 SDK (or Adafruit’s Arduino core for the BLE-only path). The Euler-update compute path is unit-tested against a reference Python implementation of Eq. (6) before on-device deployment, for both the power and HR parameterizations.

6 Research Framing

6.1 Computer Science Research Questions

1. **Algorithmic correctness under discretization.** How does Euler-step sampling frequency affect X'_{bal} accuracy relative to the continuous-time ground truth, for *both* the validated power case and the adapted HR case?
2. **Model transfer.** Does the depletion/recovery structure validated for power (Skiba et al.) produce a comparably informative fatigue signal when driven by heart rate instead — and if the two disagree, where and why? (Requires bench sessions with both a power meter and HR strap on the same athlete for direct comparison.)
3. **Concurrent firmware architecture.** What FreeRTOS scheduling policy minimizes worst-case latency while preventing sensor-buffer overflow on a single-core MCU?
4. **Extensibility toward ML (stretch goal).** Can a lightweight recurrent model trained on historical HR traces forecast near-term HR well enough to make the LED feedback anticipatory rather than reactive, addressing the cardiac-lag weakness named in Section 3.3? Full specification in Section 7.

6.2 ECE Research Questions

1. **Power budget validation.** Does measured current draw under real BLE connection and NeoPixel activity match the datasheet-derived estimate in Section 4.3, and does the system meet the 6 h runtime target with margin to downsize the battery?
2. **RF behavior in the deployed environment.** Does BLE connection stability hold up mounted

on a carbon bicycle frame or an aluminum/carbon rowing shell (both RF-unfriendly environments), and does antenna placement in the 3D-printed enclosure need adjustment?

3. **End-to-end latency budget.** With the wireless link now in the loop (strap $\rightarrow$ BLE notification $\rightarrow$ compute $\rightarrow$ LED), where does the 100 ms budget actually get spent, and is BLE connection-interval tuning sufficient to hold it?

7 Stretch Goal: On-Device HR Forecasting

This is scoped as a **Phase 5 stretch goal**, not a Phase 1–4 dependency — the core deliverable (Sections 3–6) is complete and demoable without it. It is included here because it turns the model’s stated weakness into the project’s most interesting extension, and because a real training dataset already exists (Section 7.3).

7.1 Motivation

Section 3.3 names cardiac lag as the central limitation of the HR-analog model: by the time $HR(t)$ crosses $LTHR$, the athlete’s actual effort crossed it tens of seconds earlier. The LED feedback is therefore always a step behind reality. A small forecasting model sitting in front of the depletion equation — predicting $HR(t + \delta)$ from the recent trace rather than reacting to $HR(t)$ — gives the athlete a genuine early warning instead of a delayed one, which is a materially different (and more useful) product behavior, not just a numerical refinement.

7.2 Model & Integration

- **Architecture:** single-layer GRU or LSTM, 8–16 hidden units, taking the trailing 30–60 samples of $HR(t)$ (plus first difference) at the existing 1 Hz cadence and predicting $HR(t + 5\text{--}10\text{ s})$.
- **Footprint:** at this size, a handful of hundred to ~ 2000 parameters — low single-digit kB of flash at 8-bit quantization. RAM is dominated by the inference runtime overhead (TFLite Micro’s interpreter, roughly 2 kB to 10 kB depending on configuration) rather than the model itself, and should be benchmarked against whatever the S140/S340 SoftDevice reserves before assuming it fits (Section 4.3’s RAM headroom is for the compute task, not the radio stack).
- **Inference cost:** a model this small runs in microseconds to low single-digit milliseconds on the Cortex-M4F — does not threaten the 100 ms end-to-end latency budget (Section 3.5).
- **Integration point:** the compute task (Section 3.5) gains an inference substep ahead of the Euler tick; H'_{bal} is computed against the forecast $\widehat{HR}(t + \delta)$ rather than the raw sample, and the LED hue transition fires on the forecast crossing $LTHR$.

7.3 Training Data

Unlike a typical student ML project, this does not need a bootstrapped or public dataset — historical session data already exists from AstrapheAI, Sean’s self-hosted endurance-coaching platform, which ingests Strava/WHOOP telemetry per workout including **power**, **heart rate**, **speed**, **elevation**, and additional derived fields. This is a real, already-labeled corpus of multi-athlete session data rather than a synthetic or public substitute, and is queryable directly (workout listing and windowed stream extraction) rather than requiring a new export pipeline.

- **Session split:** train/validation/test split by athlete and by session (not by sample) to avoid leaking a single ride’s autocorrelated HR trace across splits.
- **Label quality check before committing to this scope:** confirm sampling rate and gap frequency in the existing HR streams are consistent enough for a 1 Hz forecasting target, and

confirm whether per-session effort/RPE context exists or whether HR and power alone are the only signal available for training.

- **Cross-signal opportunity:** since power is also present in the same dataset for sessions where it was recorded, the same corpus supports the model-transfer research question (Section 6.1, item 2) — training and evaluating the forecast model against sessions where *both* power and HR exist gives a direct check of how much the HR-analog model’s lag actually costs, using the athlete’s own data rather than a literature estimate.
- **Offline training:** standard Python/PyTorch or TensorFlow training loop, exported and quantized to TFLite Micro (or a small hand-rolled fixed-point inference routine, given the model’s size) for on-device deployment; unit-tested against the Python reference forecast output before flashing, following the same validation pattern already used for the Euler update (Section 3.5).

7.4 Risk

Genuinely a stretch goal: if Phase 1–4 timeline slips, this is the first thing to cut without weakening the core deliverable, and the fallback in CS Research Question 4’s original framing (online $\tau_{H'}$ personalization via a lightweight estimator rather than a trained network) remains available as a lower-effort substitute.

8 Deliverables & Timeline

Phase	Milestone	Target
1	Hardware bring-up; BLE HR strap pairing verified; power budget bench-validated	Sep 2026
2	H' model on MCU; validated against Python reference and, where possible, against the power-based W' reference case	Oct 2026
3	FreeRTOS integration; UI/calibration task; latency and scheduling benchmarks	Nov 2026
4	Full feedback loop; field test with HR strap on-bike and on-water	Dec 2026
5	Formal write-up, poster, and final public demo	Apr 2027
5 [†]	<i>Stretch:</i> HR-forecasting model (Section 7) — training data audit, offline training, on-device integration	Apr 2027

[†]Stretch goal; does not block the core Phase 5 deliverable if timeline is tight.

9 Bill of Materials

Linked items are current as of this revision; prices are list price at the linked retailer and will drift. Items marked (*source locally*) are generic enough that any supplier works and are not worth pinning to a specific link.

Item	Price	Notes
Adafruit Feather nRF52840 Express	\$24.95	Core MCU. Native BLE, USB, onboard LiPo charge circuit.

Item	Price	Notes
Lithium Ion Battery — 3.7 V 2000 mA h, JST-PH	\$12.50	Plugs directly into the Feather's onboard battery port; no separate charger/BMS needed.
NeoPixel Stick — 8 x WS2812/SK6812 RGB LED	\$5.95	Primary H'_{bal} readout.
Buzzer 5V — Breadboard friendly	\$1.95	Low-reserve alert; self-driving, apply 3–5 V directly.
Tactile Button Switch (6mm) — 20 Pack	\$4.95	Up / Down / Select calibration buttons (3 used, 17 spares).
Monochrome 0.91" 128×32 I2C OLED — STEMMA QT	~\$15	Threshold-entry and live-HR display during calibration.
STEMMA QT to Premium Male Headers Cable	\$0.95	Solderless I2C hookup from OLED to Feather header pins.
USB Type C Breakout Board — Downstream Connection	\$2.95	External USB-C port (Section 4.5); wire to the Feather's Micro-USB pins.
Protoboard / perfboard (<i>source locally</i>)	~\$5	
22–26 AWG hookup wire, incl. fine 30 AWG for the USB-C piggyback joints (<i>source locally</i>)	~\$8	
JST-PH 2-pin connectors, a few (<i>source locally</i>)	~\$5	For detachable buzzer/NeoPixel leads.
Linked components subtotal	~\$69	

Optional — HR chest strap for bench testing: [Polar H10 Heart Rate Sensor](#) (~\$90–105) if the team does not already have a spare dual-mode (BLE + ANT+) strap available for dedicated bench work. Any teammate's Polar H10, Garmin HRM-Dual/Pro, or Wahoo TICKR works as-is with the BLE-only plan in Section 4.2.

10 Open Decisions & Risks

- **ANT+ vs. BLE-only:** pending confirmation that every strap the team will actually use is dual-mode. If not, the ANT+/S340 path in Section 4.2 needs to move from "documented fallback" to "Phase 1 requirement."
- **LTHR field-test accuracy:** the guided on-device test is a standard field estimate, not a lab-grade measurement; needs validation against a known value for at least one athlete before being trusted as the default calibration path.
- **Enclosure sealing:** rowing's splash exposure is a materially different environment than cycling for the button/OLED cutouts; needs a sealing approach before the on-water field test in Phase 4.

11 Advisor Engagement

Prof. Kousen's advising continues to focus on software architecture, algorithmic design, and CS rigor per the original concept summary; the hardware/ECE decisions in Section 4 are outside his stated area and are tracked here for the ECE half of the advising/grading structure. Monthly check-ins

continue to cover the written deliverables (proposal, interim report, thesis) and, per Section 6.1, the model-transfer question this revision introduces.

Contact: Sean Balbale — sean.balbale@trincoll.edu | Zoe Davidow — zoe.davidow@trincoll.edu | Kayla Walsh — kayla.walsh@trincoll.edu